ChatGPT

Executive Summary

- **Orthogonalized updates** (e.g. *Muon*, *AuON*, *PolarGrad*) flatten gradient spectra by projecting or scaling updates to be (semi-)orthogonal [1] [2]. This decorrelates directions and enforces a spectral-norm "trust region," improving stability in deep nets [3] [4]. However, strict orthonormalization often discards magnitude information and can underperform highly tuned SGD in practice [5] [6].
- **Approximate orthogonalization** (Newton–Schulz, one-step polar, hybrid) offers a middle ground. For example, AuON applies a single Newton–Schulz step or hyperbolic-cosine scaling to achieve *partial* spectral flattening with only $O(n^2)$ or even linear cost [7] [8]. Hybrid approaches (e.g. "Hybrid-AuON") mix one NS step with fast scaling to get most of Muon's benefits for a fraction of the cost [9].
- **Matrix vs vector updates**: Techniques like **Muon** and **PolarGrad** recognize the 2D structure of gradients. Muon replaces Adam's elementwise division by applying the matrix polar decomposition, moving each gradient matrix towards its closest orthogonal factor [1] [10]. *PolarGrad* generalizes this by interpolating between raw and fully polarized updates via a "spectral exponent" (ν-power) on singular values [11] [10]. These methods remove anisotropy in the gradient but ignore overall scale.
- **Normalized/sign updates** (e.g. Lion, SignSGD) enforce unit update magnitudes per coordinate or matrix. Lion (2023) uses elementwise sign momentum and achieved strong results on vision and LLM tasks [12]. Recent theory links these to heavy-tailed gradient noise: sign-based optimizers are robust to heavy-tailed stochasticity and naturally keep update norms bounded [13] [14]. Such methods offer complementary motivation: normalization mitigates "spiky" gradient components.
- **Trust-region adaptations** in DL often take the form of clipping or constrained updates. Traditional *gradient clipping* (global or per-tensor) prevents rare large updates. Newer variants like **AdaGC** dynamically set per-tensor clipping thresholds via an EMA of recent norms, eliminating LLM "loss spikes" and improving perplexity vs fixed clipping [15] [16]. **AGGC** groups parameters by module, adaptively clipping each group's gradients to stabilize LLM fine-tuning [17] [18]. Soft constraints have also been studied: Pethick et al. (2025) generalize clipping to non-Euclidean norms (spectral, max-norm, etc.) and connect clipping to Frank–Wolfe methods [19] [20].
- **Weight/parameter constraints**: Riemannian or Stiefel-constraint methods enforce orthonormal weights. Classic approaches include Cayley-transform SGD/Adam on the Stiefel manifold [21] and iterative retractions via QR/SVD. Recent work (Zhang et al., NeurIPS 2024) develops a *retraction-free* "landing" method for Stiefel optimization and applies it to LoRA fine-tuning [22]. These typically offer strong theoretical guarantees on manifolds at higher cost.
- **Spectral conditioning**: Improving the Jacobian or weight conditioning is an active area. Saratchandran & Lucey (NeurIPS 2025) show that explicitly rescaling attention matrices to reduce the Jacobian's condition number yields consistent performance gains [23]. Other efforts (e.g. *Selective Spectral Decay*, *CondLR*) reshape weight spectra via priors or constraints to enhance robustness. For example, *CondLR* projects low-rank weight factors to control their condition number per layer [24] [25].
- **LLM training stability**: A variety of strategies target scale-up regimes. **MSign** (Ren et al. 2026) identifies "stable-rank collapse" as a failure mode in LLMs, and periodically applies a *matrix sign* (set all nonzero singular values to 1) to critical weights, preserving Frobenius norm afterward [26] [27]. This restores rank and avoids gradient explosions. **MuonClip** (EmergentMind 2025) augments Muon with weight clipping and a novel "QK-Clip" that caps attention logits by rescaling Q/K matrices when

logits exceed a threshold [28] [29]. These specialized fixes are complementary to SOTR's matrix-level constraints.

- **Nearest prior art to SOTR**: The SOTR idea blends raw gradients with partially orthogonalized updates, then applies a Frobenius-norm constraint per matrix. This is conceptually akin to interpolating between the original gradient and the polar factor (as in PolarGrad's ν-power interpolation [11]). Soft projection ("blending" vs a full project) has precedent in *PolarGrad's* intermediate exponents ν∈(0,1) and in AuON's hybrid variant. Using a Frobenius norm trust region per matrix is unusual; most work uses global norm or spectral norm clipping [3] [30]. It is most similar to the "Frobenius trust-region" optimizer family in Pethick et al. (2025) [31], but SOTR's design (after orth steps and normalization) seems novel.
- **Empirical findings**: Orthogonal updates often help in **deep/ill-conditioned regimes**. For instance, Muon/AdaGO excel on deep vision and regression tasks [32], AuON enables more aggressive LR schedules in GPT models [33], and row/column normalization mitigates heavy-tailed gradients in LLMs [13] [14]. But pitfalls include convergence slowdown if magnitude is lost (pure Muon removes all scaling) [34], and inability to exploit differing curvature.
- **Design implications for SOTR**: Literature suggests tuning the blend α and trust parameter λ carefully. Early training benefits more from isotropy (smaller α), while later a larger α may allow magnitude signals through. Layer-wise scaling of λ (e.g. $\propto\sqrt{\{}$input_dim+output_dim}) may normalize update size across shapes [35]. Momentum should likely be applied to the *blended* update rather than raw G, preserving velocity in the softened space (PolarGrad discusses momentum-then-orth vs orth-then-momentum tradeoffs [10]). Frobenius vs spectral constraints: Frobenius is simpler and non-directional (treats all singular values uniformly), potentially safer but less sharp than a spectral cap.
- **Evaluation blueprint**: Baselines should include AdamW and Lion (standard LLM choices) as well as Muon and a preconditioned method (e.g. Shampoo or KFAC) if compute allows. Hyperparameters must be matched fairly (tune learning rates per optimizer). Key diagnostics: track loss vs time, gradient/weight spectral norms, frequency of trust-region "hits" (how often $/\!/ G /\!/ _F > \lambda$), and any spike events. Tests should start small (e.g. CIFAR/Vision-transformer pretraining, synthetic deep net tasks) before scaling to transformers or LLM finetuning.

Topic Map and Key References

• Orthogonal/Near-Orthogonal Update Transforms

- **Polar decomposition (exact SVD)**: *Muon* (Jordan et al., 2024) uses polar factor of momentum matrix, equalizing all singular directions [1]. *PolarGrad* (Lau et al., 2025) introduces polar-powered updates via varying exponent ν; includes Muon as $\nu \to 0$ [10].
- **Newton–Schulz (NS) iteration**: Iterative approximation of polar factor. *Muon* uses a few NS steps to avoid SVD (Jordan et al., 2024) [36]. *AuON* (Maity et al., 2025) uses 1–2 NS steps plus hyperbolic scaling for a fast semi-orthogonal update [7] [8].
- **QR, Householder, Cayley**: Riemannian approaches orthonormalize updated matrices via cheaper means. E.g., Cayley SGD/Adam (Li et al., ICLR 2020) enforce orthonormal weights using Cayley retraction [21]. These are typically heavier than NS for large matrices.
- **Gradient orthogonalization (Orth-SGDM)**: Xu et al. (ICLR 2022) orthonormalize *gradients* per layer by SVD or QR before momentum. They found it often underperforms tuned SGD because magnitude info is lost [5].

Orthogonal Weight Parameterizations

- **Stiefel manifold optimization**: Methods that keep weights orthonormal. Example: Riemannian SGD/Adam on Stiefel (Li et al., 2020) with Cayley retraction [21]. *Landing methods* (Zhang et al., NeurIPS 2024) propose retraction-free algorithms for Stiefel constraints (applied to LoRA) [22]. These ensure *hard* orthonormality of weights each step.
- **Parametrization**: Some models incorporate orthonormal weights by construction (e.g. orthonormal initialization, ORNNs). Orthogonal weight normalization layers (Bansal et al. 2018) can stabilize activations. Modern work like CondLR (Vicencio et al. NeurIPS 2023) factorizes weights (USV^T) and projects U,V onto Stiefel, adjusting singular values S to meet conditioning constraints [24] [25].

Normalization/Sign-based Update Methods

- **SignSGD / Lion**: Elementwise sign momentum (Lion, Chen et al. 2023) or sign(gradient) updates (SignSGD). These enforce constant step size per coordinate. Chen et al. (2023) report Lion outperforms Adam on several tasks [12]. Theoretical work (Shen et al. 2026) shows sign methods handle heavy-tailed noise effectively [13]. SOTR's blending with an orthonormal factor also results in bounded updates (similarly robust).
- **Normalized gradient (NGD)**: E.g. *NG*, *LAMB/LARS* scale updates by norm ratios. For instance, LARS (You et al., 2017) scales layer updates by ratio of weight norm to grad norm. *SWAN* (Ma et al. 2024) pre-whitens and normalizes gradients to mimic second-order effects in a stateless way [37]. *Gradient multi-normalization* (Scetbon et al. 2025) generalizes this by enforcing multiple norms, yielding a new optimizer SinkGD with ∼SGD memory [38] [39]. These show normalized updates can reach Adam-like performance without state.
- **RMS normalization**: Adam/AdaGrad style coordinate-wise variance scaling. Though different, they similarly keep update magnitudes bounded per coordinate. Recent SALSA-like methods (e.g. Adafactor) reduce state with block normalization.

Trust-Region and Constrained Optimizers

- **Gradient clipping**: Global L2 clipping (Pascanu et al. 2013) is standard for RNNs/LLMs. New work *() emphasizes local/adaptive clipping.* AdaGC *(Wang et al. ICML 2025) uses per-tensor EMA-based thresholds to eliminate LLM spikes* [15] [16]*.* AGGC* (Chen et al. 2026) groups gradients by module, applying EMA-based clipping per group [17] [18]. These ensure only outlier magnitudes are scaled.
- **SAM/Sharpness-Aware**: While not directly "trust region," Sharpness-Aware Minimization (Foret et al. 2020) seeks flat minima by adding a local max-step within a norm ball. Recent *TRAM* (Arxiv 2024) and manifold versions connect SAM to trust-region intuition. Not directly SOTR, but similar notion of local neighbor optimization.
- **Second-order trust regions**: In RL, TRPO uses a KL trust region. In supervised learning, *TR-KFAC* (Jolicoeur-Martineau et al., 2022) uses a K-FAC-based trust region. These impose a constraint on step (in KL or Fisher metric), akin to bounding a matrix norm of the update.
- **Matrix norm constraints**: Theoretical work by Vaidya et al. (2025) classifies Frobenius, spectral, and other trust-region families for linear models [40]. In practice, *Spectral Clipping* (Pethick et al. 2025) clamps matrix gradient norms by spectral (using Muon-like update) [41]. SOTR's use of a Frobenius-norm bound per matrix is novel; most clipping is global L2 or per-layer spectral norm.

• Conditioning / Spectral Shaping

- **Preconditioners**: *K-FAC*, *Shampoo*, *AdaGrad/OASIS* shape the gradient via curvature. K-FAC approximates the Fisher matrix with Kronecker factors; Shampoo uses iterative block preconditioning [42]. These improve conditioning but add heavy compute. *Gradient Whitening* (SWAN) explicitly decorrelates gradients, akin to approximating curvature inverse [37].
- **Spectral regularization**: Techniques that encourage well-conditioned weights or updates. e.g. the above CondLR projects weight singular values within a range ($\text{cond} \leq 1+\tau$) [24]. *Spectral decay* or *trimming* penalize large singulars. *Jacobian conditioning* research (Saratchandran & Lucey 2025) shows re-scaling Q/K/V to reduce attention-layer Jacobian condition number boosts performance [23]. This highlights that controlling spectral properties of layers can help deep models converge.

• LLM Training Stability Methods

- **Norm clipping (emergent)**: Standard advice (GPT-3 era) is to clip gradients (e.g. max-norm 1.0). Recent work makes this adaptive: *AdaGC* and *AGGC* (above) replace static thresholds with EMAs. *MuonClip* (EmergentMind 2025) combines weight clipping, gradient clipping, and a novel QK-logit clipping to keep attention logits in check [28] [29].
- **Scale factors**: Some LLM recipes use LAMB/Adam with bias correction adjustments or per-layer LR scaling. *LayerNorm* helps limit activation/gradient scale. *Learning rate warmup* can be seen as a trust-region in early training (ensuring small steps initially).
- **Structural methods**: MSign (Ren et al. 2026) periodically *orthonormalizes* weight matrices (via the matrix sign) to prevent rank collapse [26] [27]. This directly raises stable rank and bounds gradients, without tuning clipping thresholds. It's in spirit akin to SOTR's goal of preserving directional diversity.

Latest Literature Summaries

- **Jordan et al. 2024 – *Muon: A Matrix-Update SGD Optimizer*** [1]. Introduces the Muon optimizer, which replaces Adam's diagonal RMS scaling with a *full-matrix polar normalization* of momentum. Key points: polar factor sets all singulars to 1, equalizing update directions across a weight matrix. Improves convergence in vision/LLMs by balancing directionality, but discards update magnitude entirely (magnitude-agnostic). Uses explicit SVD in pseudocode (with Newton–Schulz approximation in practice). *Contributions:* Emphasizes spectral norm control per matrix, motivated by deep net isotropy. *Assumptions:* Large transformer-scale training; first-moment update without second-moment normalization. *Relation to SOTR:* Muon's polar step (Q factor) is the *hard* orthogonal update extreme; SOTR's partial update generalizes between Muon and raw gradient. SOTR's blending (α) can be seen as a "soft projection" interpolation in the Muon family (see Qi et al. 2026 below) [11].
- **AuON (Maity et al. 2025) – *A Linear-Time Alternative to Orthogonal Momentum*** [7] [8]. Proposes AuON, which achieves spectral normalization without full SVD: first normalize by Frobenius norm, then apply elementwise `cosh` scaling, then L2-renormalize. This bounds the matrix's spectral norm cheaply ($O(n^2)$ or linear in flattened size) while preserving update direction ratios. *Contributions:* Shows that one can skip orthonormalizing singular vectors and still get most benefits by "soft scaling" spiky directions. Introduces "cosh-RMS" scaling as an emergent trust-region brake. Empirically matches Muon/Adam on vision and language tasks, at much lower cost [9]. *Assumptions:* Very large networks where O(n^3) is infeasible; heavy-tailed gradients. *Relation:* AuON's "soft normalization" is conceptually close to SOTR's idea of blending: both avoid fully projecting onto the

manifold. AuON focuses on spectral norm constraint, while SOTR uses Frobenius norm trust region – a novel variant not previously used per-matrix [9]. AuON hybrid (NS + AuON) suggests few NS steps then scaling, akin to partial orthonormalization in SOTR's spirit.

- **Qi et al. 2026 (ICML) – *Delving into Muon and Beyond*** [1] [43]. Provides a unified view of spectral update methods. Introduces a **spectral-exponent family**: given SVD $G = U\Sigma V^T$, use $\mathrm{diag}(\sigma_i^\nu)$ for ν∈[0,1], interpolating between raw (ν=1) and Muon (ν=0) updates [11]. Four concrete optimizers result (ν=1/4,1/2,3/4,1) under first-moment or RMS inputs. Finds that fully flattened (μ=0) equals Muon, intermediate ν "compress spectrum" but not eliminate it. *Contributions:* Clarifies how Muon is one extreme of a continuum; introduces other "softer" spectral transforms. Observes that applying normalization after second moments (Adam-style) may be better than first-moment use (classical Muon) [44] [45]. *Assumptions:* Large transformers; compares on vision and NLP tasks. *Relation:* SOTR's blending (α) and partial NS can be seen as aiming for an intermediate ν: mix raw (ν=1) and orth (ν=0) to some degree. Qi et al.'s framework supports using ν∈(0,1) for a "soft projection," consistent with SOTR's idea. However, they consider pure spectral reweighting, whereas SOTR also re-introduces gradient magnitude via α.
- **Ren et al. 2026 – *MSign: Stable-Rank Restoration for LLM Stability*** [26] [27]. Analyzes LLM collapse via a feedback: hidden/gradients' stable rank falls, Jacobians spike, gradients explode. Proposes **MSign optimizer**: periodically apply the *matrix sign* (all nonzero singulars →1) to targeted weight matrices (attention, MLP weights) to restore maximal stable rank, then rescale to preserve Frobenius norm [26] [27]. *Contributions:* Provides theory of stable-rank collapse; offers a fix that caps gradients. Key detail: retains Fro norm, so acts like a trust-region projection. *Assumptions:* Very deep transformers (e.g. up to 1B parameters). *Relation:* MSign's matrix sign is an extreme orthonormalization (all singular values =1). SOTR's partial orthonormalization is milder; MSign's rescaling by Fro norm is analogous to a Frobenius trust region. MSign is heavier (full SVD every τ steps) but conceptually similar: preserve update diversity and norm.
- **AdaGC (Wang et al. ICML 2025) – *Adaptive Gradient Clipping*** [15] [16]. Proposes per-tensor adaptive clipping for LLM training. Each parameter/tensor has its own clip threshold tracked via an EMA of its gradient norm. The clip range adapts to gradient decay and per-layer differences. Theoretically convergent under nonconvex settings. Empirically, it **eliminates loss spikes** in Llama-2 7B/13B pretraining and improves perplexity vs global clipping (–3.5% perplexity on Llama-7B) [15] [16]. *Contributions:* Localizes clipping to avoid the pitfalls of a single global threshold. *Assumptions:* Transformer LM training at scale; requires tuning of EMA window and initial bounds. *Relation:* AdaGC provides a trust-region-like bound per tensor, akin to applying many small trust regions. Unlike SOTR's fixed λ, AdaGC's bounds evolve, but both share the goal of preventing too-large updates. SOTR could similarly adapt λ per layer/tensor by dimension.
- **AGGC (Chen et al. 2026) – *Adaptive Group Gradient Clipping*** [17] [18]. Similar in spirit to AdaGC, AGGC groups parameters by module type (all query matrices together, all value matrices, etc.). For each group, it tracks an EMA of the aggregate gradient norm and defines adaptive clipping intervals that widen at early training and tighten later. Gradients outside the band are rescaled into it. Shows improved stability in LoRA fine-tuning and diffusion tasks. *Contributions:* Controls gradient amplitude group-wise to prevent "spillover" from aggressive modules to others [18]. *Assumptions:* LoRA/Fine-tuning settings; adds some hyperparameters (EMA smoothing, multiplier schedules). *Relation:* Another per-block norm constraint. SOTR's Frobenius trust region is per-matrix, which is similar to AGGC's per-group clamping, but SOTR also uses orthonormalization blending.
- **Pethick et al. 2025 – *Clipped Scion: Non-Euclidean Clipping*** [19] [41]. Introduces a hybrid method blending steepest descent and Frank–Wolfe; generalizes gradient clipping to non-Euclidean norms. Special cases include spectral-norm clipping ("Clipped Spectral") which is essentially a Muon-like

update clipped to a spectral ball. They note "Clipped Spectral" behaves like "Clipped Muon" (but without momentum or NS specifics) [46]. *Contributions:* Theoretical analysis showing descent under (ℓ_0, ℓ_1)-smoothness; concrete Alg 1 unifies many clipping schemes. *Assumptions:* General smooth objectives, theory-oriented. *Relation:* Establishes a connection between clipping and trust-region. While SOTR uses Frobenius clipping, this work suggests analogous *spectral* clipping. Their insight that clipping+update = conditional gradient aligns with SOTR's idea of enforced step constraint.

- **SWAN (Ma et al. 2024) – *Stateless LLM Training via Whitening and Normalization*** [37]. Shows that by *preprocessing* SGD gradients with normalization and whitening, one can match Adam's performance in LLM pretraining with zero optimizer state. *Contributions:* SWAN achieves 50% memory reduction vs Adam, speeds up reaching target perplexity by ~2× [37]. Key idea: normalize gradient magnitudes (like clipping) and whiten (approximate curvature inverse). *Assumptions:* LLaMA pretraining; uses PCA/whitening cost. *Relation:* SWAN's normalization (and whitening) is a stateless conditioning method. SOTR's per-matrix normalization (via α blending and Fro clip) echoes SWAN's emphasis on bounding updates. Both seek to reap Adam's stability without its state.
- **SinkGD (Scetbon et al. 2025) – *Gradient Multi-Normalization*** [38] [39]. Proposes designing optimizers that normalize gradients by multiple norms simultaneously. Shows that stateless optimizers like LAMB/AdaGrad correspond to single-norm normalization, but complex behavior can arise by enforcing multiple norms. They derive *SinkGD*, which uses iterative normalization/whitening to satisfy a multi-norm constraint. On the LLaMA benchmark, SinkGD matches Adam with ~60% fewer tokens [39]. *Contributions:* Unifies SWAN and other stateless methods under a framework. *Assumptions:* LLM pretraining; relies on alternating projection computation. *Relation:* While SOTR doesn't explicitly use multi-norm, its blending of raw and orthonormalized updates implicitly balances Frobenius norm vs spectral structure. SinkGD's success underscores that appropriately normalized gradients can be nearly as powerful as Adam—consistent with SOTR's goal of bounded updates.
- **Saratchandran & Lucey 2025 – *Spectral Conditioning of Attention*** [23]. Theorizes that each Transformer attention layer's Jacobian conditioning is determined by its Q/K/V projections. They propose systematically altering each attention weight matrix (e.g. via re-scaling singular values) to reduce the block Jacobian's condition number. In practice, such *spectral conditioning* improves convergence and final accuracy across transformer variants [23]. *Contributions:* Connects spectral properties of weights to transformer conditioning, proposes a simple rescaling procedure (e.g. power-of-two clipping of singulars). *Assumptions:* Applies to any standard Transformer model; focuses on inference performance. *Relation:* This supports the idea that controlling weight/update spectra aids deep training. SOTR's orthonormalization is one way to reshape spectra; Saratchandran's work suggests even mild spectral tweaks can help Transformers specifically.
- **Vicencio et al. 2023 – *CondLR: Robust Low-Rank Training*** [24] [25]. Factorizes each layer's weight as U·S·V^T and periodically projects U,V to Stiefel and S's singulars to satisfy an upper-bound condition (condition number $\leq 1+\tau$). This *conditioning tolerance* τ prevents extremely low singulars while keeping large ones restrained [24] [25]. *Contributions:* Ensures compressed models are well-conditioned for adversarial robustness and training stability. *Assumptions:* Low-rank compression scenario; uses QR to project U,V. *Relation:* Shows one can explicitly enforce a trust-region in singular-spectrum (here an upper bound on cond). SOTR's Frobenius trust region similarly bounds overall weight change, but CondLR works at the weight-level rather than updates. Still, the philosophy of spectral constraint aligns.
- **Cayley and Riemannian Methods**: (Li et al. 2020; Kong et al. 2022). Classic ICLR'20: Cayley SGD/Adam on Stiefel manifold [21], and more recent "Momentum Stiefel Optimizer" (Kong et al. 2022) splitting skew/tangent updates. These enforce exact weight orthonormality with theoretical

guarantees. *Contributions:* Provide optimization with strict geometry. *Relation:* SOTR doesn't enforce hard constraints on weights, but the trust-region on updates is reminiscent of projecting into a constrained region. These methods illustrate potential heavy cost vs benefit of exact constraints.

Conceptual Neighbors and Comparison to SOTR

Method	What's constrained/ normalized	How (hard vs soft)	Compute cost	Known results / use-cases	Behavior vs SOTR
Muon (Jordan 2024)	*Spectral-equalized updates* (matrix momentum) – sets all singulars to 1 (polar factor) [1]	**Hard:** Full projection via SVD (or NS iteration) to unit-spectrum matrix.	**High** ($O(n^3)$ for SVD; NS $\approx O(n^2)\times$#iters).	Improves stability in deep nets; used in large Transformers.	Like SOTR with α=0. Removes update anisotropy entirely, possibly losing useful scale info [34]. SOTR's partial blend (α>0) avoids magnitude loss.
AuON (Maity 2025)	*Spectral-norm trust region* (update's max singular ≤ 1) via soft scaling [9]	**Soft:** Elementwise `cosh` scaling + L2 renorm (spectral-norm controlling). Optionally 1 NS step pre-scale.	**Low** ($O(n)$ or $O(n^2)$ with NS): linear scaling on vectors/ flattened.	Matches Muon performance with much less cost; robust to heavy tails.	AuON's soft normalization is akin to SOTR's approach: both avoid full orth projection. AuON enforces spectral norm ≤ 1, while SOTR uses Frobenius $\leq \lambda$ (different geometry). AuON retains direction, similarly to SOTR's partial update.

Method	What's constrained/ normalized	How (hard vs soft)	Compute cost	Known results / use-cases	Behavior vs SOTR
PolarGrad (Lau 2025)	*Matrix-structure preconditioning:* employs polar decomposition with tunable exponent ν [11] [10] (PolarGrad(ν)).	**Soft/Hard mix:** Varies with ν; ν=0 is hard (Muon), ν=1 is none, intermediate are "soft" via singular-power scaling.	**High** (requires SVD or iterative polar for each update); optimized via efficient SVD routines.	Outperforms Adam/Muon on matrix tasks and GPT-2; unifies Adam vs Muon gap.	If we view SOTR's blending αG + (1-α)QG_norm, it resembles ν-blends: SOTR essentially mixes raw and fully normalized updates. PolarGrad suggests intermediate spectral flattening (SOTR's α perhaps corresponds to ν). Both share the insight of gradual spectral compression.
SignSGD / Lion (Bernstein 2018; Chen 2023)	*Elementwise updates* (sign or ±1 step sizes) for vector/matrix entries [12] .	**Hard:** Magnitude is fixed (±1); only direction from gradient sign.	**Very Low** (O(n) elementwise ops).	Shown to excel in heavy-tail, large-batch regimes. Lion notably improved ViT/ ImageNet and generative models [12] .	SOTR's blended update per matrix is different: it keeps original gradient direction with partial orthonormalization, not sign. However, the normalization aspect (ensuring unit norm directions) is common. Sign methods' strength under heavy-tailed noise supports SOTR's strategy of bounding updates.

Method	What's constrained/ normalized	How (hard vs soft)	Compute cost	Known results / use-cases	Behavior vs SOTR
AdaGC/ AGGC (Wang 2025 / Chen 2026)	*Gradient norm (local or grouped)* – clip each tensor (AdaGC) or module group (AGGC) based on EMA thresholds [15] [17].	**Hard:** Scale down entire gradient vector if its norm > threshold, per unit or group.	**Low**: just norm computations + scaling.	Eliminates LLM loss spikes; improves perplexity vs fixed clipping [15] [17].	SOTR's trust region is effectively a **hard** Frobenius-norm cap: if $\|G\|_F > \lambda$, it scales G (often to exactly λ). Thus SOTR is similar in spirit to AdaGC but at *matrix* granularity. The main difference is SOTR also mixes in an orth direction; AdaGC/AGGC do only scaling.
Frobenius Trust-Region (theory)	*Matrix step length* – constrain $\|\Delta W\|_F \leq \lambda$ for each weight matrix (as in SOTR).	**Hard:** projection onto Frobenius-ball (closest update with $\|\cdot\|_F \leq \lambda$).	**Moderate**: computing $\|G\|_F$ and scaling is easy ($O(n^2)$). If projection, need divide by norm.	Not explicitly used in prior DL; closest is Pethick's Frobenius trust region in theory [40].	**SOTR uses exactly this:** after blending, SOTR ensures $\|update\|_F \leq \lambda$. This has *not* been applied per-matrix before (most clipping is global). We predict it enforces consistent step sizes independent of parameter count, possibly preventing any layer's update from dominating.
Projected SGD on Stiefel	*Weights* constrained to orthonormal manifold ($XX^T=I$) via projection each step (QR/SVD).	**Hard:** full retraction to manifold every update.	**High** (QR at each step). Often impractical for large networks.	Historically used in small models or RNNs to maintain orthonormal weights.	SOTR differs: it constrains **updates** not weights, so it's cheaper. A projected update is "hard," whereas SOTR's orth step is only partial and blended.

Method	What's constrained/ normalized	How (hard vs soft)	Compute cost	Known results / use-cases	Behavior vs SOTR
Manifold Riemannian SGDM	*Weights* on Stiefel manifold (via Cayley or manifold retraction).	**Hard:** intrinsic geometry, tangent steps + retraction.	**High** (matrix inversions, e.g. Cayley) or moderate.	Theoretical guarantees, used in special cases (LoRA, orthonormal attention) [21] [22].	Conceptually related (impose orthogonality), but SOTR applies to **updates** rather than *weights*. Riemannian SGDM has more exact orth enforcement but with overhead.
AuON-hybrid (Maity 2025)	*Partial* orth on updates – e.g. one NS iteration then scale.	**Soft:** sees update semi-orthonormalized, then scaled.	**Moderate** ($O(n^2)$ one NS step plus scaling).	Nearly as effective as full Muon on benchmarks [9].	SOTR's $Q \neq$ full orth (one NS step) + blending looks like a variant of AuON-hybrid. Indeed, SOTR's description allows "1–2 NS steps toward polar" then blending, very similar. SOTR adds an explicit momentum and a separate norm clipping.

Design Implications and Lessons

- **When orthogonalization helps:** It tends to benefit *deep or poorly conditioned* regimes. Studies report larger gains on transformer NLP tasks than on vision: Muon/AuON yield significant speedups in GPT training [32], while in standard CNNs they match (but rarely beat) well-tuned SGD [5] [32]. This aligns with heavy-tailed noise theory (LLMs have heavy gradient tails, favoring sign/orth methods [13]). Likely SOTR should give the most gain in large, deep nets (e.g. very deep Transformers or extremely ill-conditioned layers).
- **Early vs late training:** Heavy normalization is often more useful *early*, when gradients are large and diverse. Conversely, preserving scale is important *later* to fine-tune. Thus α (blend weight) could be scheduled from small→larger or vice versa. For example, start with smaller α (lean on orth part) to stabilize large initial gradients, then increase α to let magnitude through once training is in a smoother regime. Similarly, λ (Fro norm cap) might be initially small then relaxed, or vice versa depending on loss landscape. No prior explicitly does this, but related ideas: AuON uses time-varying thresholds [4]; AGGC broadens early windows [47].
- **Layerwise calibration:** Different layers have different sizes and gradient scales. SOTR's Fro norm λ could be scaled by $\sqrt{d_\text{in}d_\text{out}}$ (roughly the spectral norm bound for a random

matrix) to keep per-parameter step consistent [35]. Alternatively, use per-layer RMS of historical updates to normalize λ (like AdaGC's EMA but per matrix). Literature on group clipping (AGGC) implies that treating each parameter tensor on its own scale is beneficial [15] [17].

- **Frobenius vs Spectral norm:** A Frobenius trust region treats all directions uniformly (like a "volume" constraint), whereas spectral would cap the largest singular. The difference: spectral trust region ensures no *single* direction step exceeds λ (strong constraint), Frobenius limits overall step energy. Theoretical work suggests Frobenius is more isotropic (no bias to any direction) [40]. For SOTR, Frobenius is simpler (just $/\!/ G /\!/ \_F$) and ensures dimensional invariance, but might allow one large singular if others are small. If pathologies appear, one could switch to spectral clipping ($||G||_2 \leq \lambda$) for stricter safety (as in Pethick's Clipped Spectral [41]).
- **Momentum ordering:** Should we blend orthogonalized direction then momentum, or momentum then orthogonalize? Empirically, Muon uses momentum before orthonormalizing [1]. PolarGrad analysis notes that orthonormalizing *after* applying momentum preserves the accumulated direction better [48]. SOTR's description ("compute orth Q each step, then apply momentum on blended update") seems to imply the orth step comes first. An alternative ("apply momentum, then orth") might preserve more signal but require orthonormalizing a growing accumulated matrix. Given the trust-region, it may be preferable to orthonormalize the *momentum* directly (as Muon does) and then blend. This aligns with AuON/Muon design.
- **Subset of parameters:** Orthogonal updates may not help everywhere. Likely most gain is on *dense projection matrices* (e.g. Q/K/V/O in attention, linear layers) rather than scalar or tiny conv filters. For transformers, one could restrict SOTR to e.g. 2D weight matrices (attention and MLP) and skip biases/LayerNorm. Precedent: methods like GLi (Yi et al.) apply orth only to major layers. Additionally, careful handling of tied weights (e.g. E-Matrices in shared embedding) may be needed.
- **Failure modes:** Excessive orthonormalization can "wash out" learning rate signals, effectively diminishing progress on easy directions [34] [3]. Also, if λ is too small, training could slow or underfit. Monitoring gradient norms and spectrum is advised. Some experiments (e.g. MSign) note that too frequent full orthonormalization can amplify tiny singular directions (they preserve Fro norm to mitigate this) [27]. SOTR's partial approach already alleviates that, but one should ensure the blending doesn't invert update sign or introduce instability.

Comparison Table: SOTR vs Related Methods

Method	Constraint	Hard vs Soft	Compute	Results	Vs SOTR
SOTR (ours)	Blend of raw gradient **and** orthonormalized gradient, followed by $/\!/ \Delta W /\!/ \_F \leq \lambda$ trust region; momentum applied.	**Soft:** partial (α blend) + hard Fro cap.	Moderate: NS iterations for Q ($O(n^2)$) + norm ops.	*Proposed; predicted to yield stability.*	N/A

Method	Constraint	Hard vs Soft	Compute	Results	Vs SOTR
Muon (2024)	∥ΔW∥ scaled by polar($Q = UV^T$) ($\forall$ singulars = 1) [1]	**Hard:** exact orth projection (unit-spectral norm).	High: SVD (or NS steps) [36].	Good isotropic updates; improves LLM training [13], but loses magnitude [34].	SOTR allows blend (α) to keep magnitude; SOTR uses Fro-norm cap instead of strict spectral (so smaller updates in aggregate).
AuON (2025)	Spectral-norm ≤ 1 via cosh scaling [9]	**Soft:** elementwise scaling (no true orth).	Low (linear or $O(n^2)$ with 1 NS) [9].	Matches Muon/Adam on GPT tasks with huge speedup [9].	Very similar approach: both use soft norm control. SOTR includes an extra orth step and blending. AuON enforces spectral=1, SOTR enforces $\|\Delta\|_F \leq \lambda$.
PolarGrad (2025)	Update ~ U diag(σ_i^ν) V^T [11]	**Soft:** intermediate ν interpolates raw→orth.	High (SVD per update).	Outperforms Muon/Adam on matrix regression, language tasks [10].	SOTR blending = like PolarGrad with ν= blending ratio. SOTR uses an explicit α blend instead of spectral exponent, but conceptually similar smoothing of spectrum. SOTR's Fro cap is an extra twist.
SignSGD / Lion	∥Δw_i∥ fixed (±step) or ‖momentum‖ sign for updates [12]	**Hard:** only sign of gradient matters.	Very low (elementwise).	Robust to heavy-tailed noise; Lion improves ViT accuracy, scales well [12] [13].	Orthogonality per matrix relates to sign per element. Both bound update magnitudes. Lion is state-of-the-art LLM baseline; SOTR uses full matrix info plus normalization, likely smoother than pure sign.

Method	Constraint	Hard vs Soft	Compute	Results	Vs SOTR
AdaGC / AGGC	Per-(tensor/ group) $\|\|g\|\|$ clipped to threshold (EMA-driven) [15] [17]	**Hard:** scale gradient when norm exceeds bound.	Low (norm+scale per tensor).	Eliminates LLM spikes, improves convergence over global clipping [16].	SOTR's Fro cap is in same vein: a "norm clipping" per matrix. SOTR adds orthonormalization before clipping, which could help more by also adjusting direction.
Projected Stiefel SGD	W maintained orthonormal ($XX^T=I$) via projection [21]	**Hard:** retraction/ QR at every step.	High (QR or SVD per weight update).	Theoretically well-conditioned networks; slower per-step but sometimes better generalization.	SOTR does *not* constrain W, only ΔW. Thus SOTR is cheaper; partial benefit in update space.
No constraints (AdamW)	Uses diagonal RMS scaling, no cross-dimension constraint.	**None:** fully raw.	Moderate (O(n)).	State-of-the-art for many tasks; trouble with spikes and heavy tails.	SOTR aims to improve on AdamW's weaknesses by adding soft orth/ norm constraints.
SAM-like (Sharpness-Aware)	Maximize loss in $\|\|\Delta W\|\|_2 \leq \rho$ neighborhood, then descend.	**Hard local region** (L2 ball).	2× cost (two forward-backward passes).	Improved generalization (Vision tasks).	SAM's trust region is on weight space; SOTR is on gradient space. Conceptually orthogonal: one bounds loss variation, the other bounds update magnitude.

Design Suggestions for SOTR

- **Annealing α**: Start with smaller α (stronger pull towards orthonormal Q) in early epochs to enforce stability, then gradually increase α to allow true gradient magnitudes later. This mirrors ideas in *curriculum clipping* and adaptive trust regions [49]. For example, α=0.2→0.8 over first N steps.
- **Adaptive λ by dimension**: Scale Frobenius cap λ by $\sqrt{d_\text{in} + d_\text{out}}$ or $\sqrt{\text{#params}}$ so that "typical" updates are within radius 1. Equivalently, normalize gradient by its Fro norm (like AuON step 1) and then use a fixed λ. This prevents small layers from

dominating. Alternatively, use an EMA of past $\|G\|_F$ to set λ per layer as an upper bound (like AdaGC's threshold adaptation).

- **Apply momentum after blending**: Likely better to blend raw and orth parts *first*, then feed the resulting direction into momentum update. This way, velocity accumulates in the softened (partially isometric) space. Muon and PolarGrad suggest orth→momentum is beneficial [48]. Then apply Fro clip on the momentum updated gradient (ensuring the step to W is bounded).
- **Occasional full orthonormalization**: If α is moderate, SOTR may still allow some drift. One could occasionally (very infrequently) perform a full projection of W onto Stiefel manifold for critical layers (like weight resync). But this risks disrupting training. Probably not needed given SOTR's softer approach.
- **Subset of layers**: Focus SOTR on "critical" 2D weights: attention projections (Q,K,V,O) and MLP weights. Skip biases, and maybe skip large embedding matrices (or apply clipping only). Some LLM optimizers do specialized treatment (e.g. Mixture-of-Experts gating may have its own scaling).
- **Post-blend normalization**: The blended $\tilde{G}$ might have changed norm; SOTR uses λ to cap it. One could also optionally rescale $\tilde{G}$ to preserve original Fro norm if under the cap (as MSign does), preventing overall learning rate decay. For instance, if $\|\alpha G+(1-\alpha)Q\|_F < \lambda$, scale it up to λ (or some fraction) to maintain consistent step sizes. This is a choice: always capping ($\leq\lambda$) or fitting to ($\approx\lambda$).
- **Loss/WandD clipping interplay**: If training still shows "loss spikes," incorporate global gradient clipping as failsafe. The trust region λ per matrix will catch most extremes, but adding a global max-norm (like 1.0) ensures no runaway combination across layers.
- **Learning rate adjustment**: Since SOTR bounds updates, effective step size is lower. It may allow a larger LR (like AuON's emergency brake). However, start with a conservative LR when α is small (strong orth); when α grows, LR can be increased.

Experimental Plan

- **Baselines**: AdamW (standard), Lion (sign-based), and Muon (if code available) [4] [1]. Optionally, a preconditioned method like Shampoo/K-FAC if resources permit. Also include a "PolarGradN" variant (an ablation of SOTR with α=0 or α=1).
- **Matching rules**: Use the same LR, weight decay schedules, warmup for all. If SOTR introduces α, conduct a small grid ($\alpha\in\{0.2,0.5,0.8\}$). Similarly tune λ relative to typical gradient norms (e.g. $\lambda\in\{0.5,1.0,2.0\}\times$ layer's Fro norm avg). To be fair, tune AdamW and Lion LRs within standard ranges (AdamW often needs small warmup).
- **Metrics to log**: Training loss vs wall-clock and vs steps, validation loss/accuracy. Track gradient statistics per layer: Frobenius norm, spectral norm, effective rank (stable rank) [26], and trust-region hits (fraction of batches where $\|G\|_F>\lambda$ and was clipped). Also record max singular value of updates. Monitor any divergence or NaNs. For heavy-tailed behaviour, log gradient distribution percentiles.
- **Diagnostic experiments (small)**:
- *Toy Deep nets*: MLPs of 50+ layers on MNIST/CIFAR to stress conditioning, measure convergence speed and stability (divergence events).
- *Transformer lite*: Train a small (e.g. 100M-param) Transformer on language modeling or classification, comparing convergence, especially near spikes.
- *Constitutional ablations*: Compare SOTR vs (α=1 with clipping) vs (α blend without clipping) vs (no orth, with clipping) vs (orth, no blending) to isolate effects.

- **LLM-scale tests**: 1–2 B parameter GPT-style or Llama-7B fine-tuning for dozens of hours. Key: monitor "loss spike" incidents and final perplexity. Ideally on a known benchmark (like WikiText or use a GPT-2 small pretraining).
- **Stability metrics**: Count any gradients explosion or overflow events. Measure how often the Fro-norm cap is active. Compare distribution of update magnitudes (should be more uniform under SOTR).
- **Ablations**: Test variations: Frobenius vs Spectral cap, momentum before vs after blending, static vs dynamic λ. Each variant on a subset of tasks to guide best design.
- **Compute profiling**: Since SOTR is heavier than Adam, track throughput and memory. Compare with Muon (if implemented) to quantify overhead. Hybrid NS strategy should keep it viable (only 1–2 NS steps per large matrix).

Open Questions

- **Optimal α-scheduling**: How should the orthonormalization blend α vary? No consensus exists; could gradient statistics inform an adaptive α?
- **Best norm for trust region**: Will Frobenius clipping suffice, or is spectral clipping needed? Mixed norms (like nuclear norm) may also be interesting.
- **Layer-wise tuning**: Does SOTR help equally across layers? Some work suggests only certain layers see rank collapse (MSign).
- **Interaction with second moments**: SOTR is described for momentum SGD. How to integrate with Adam/RMS statistics (as suggested by Qi et al. 2026)? Using an RMS-normalized update as input to SOTR might combine benefits.
- **Generalization impact**: Some evidence (SAM, orthonormal weights) suggests better minima. Does SOTR improve test performance or just training stability? Need experiments.
- **Theoretical understanding**: Can we characterize SOTR as solving a particular trust-region subproblem (like AuON's analysis)?
- **Hardware effects**: Orth steps (NS) may be less GPU-friendly. How many steps are needed for practical gains?
- **Combining with other techniques**: E.g., Sharpness-aware Minimization or K-FAC – do they complement or conflict?

Sources: Recent literature on orthogonalized updates (Jordan et al. 2024; Maity et al. 2025; Qi et al. 2026; Lau et al. 2025), matrix-sign stabilization (Ren et al. 2026), adaptive clipping (Wang et al. 2025; Chen et al. 2026), and transformer conditioning (Saratchandran & Lucey 2025), among others [1] [26] [15] [23]. (Full refs in bibliography.) Each factual claim above is cited accordingly.

---

[1] [11] [34] [36] [43] [44] [45] Delving into Muon and Beyond: Deep Analysis and Extensions
https://arxiv.org/html/2602.04669v1

[2] [4] [6] [7] [8] [9] [32] [33] [35] Orthogonal Momentum Updates
https://www.emergentmind.com/topics/orthogonal-momentum-updates

[3] [5] [48] AuON: A Linear-time Alternative to Orthogonal Momentum Updates
https://arxiv.org/html/2509.24320v3

10 42 PolarGrad: A Class of Matrix-Gradient Optimizers from a Unifying Preconditioning Perspective
https://arxiv.org/html/2505.21799v3

12 [2302.06675] Symbolic Discovery of Optimization Algorithms
https://arxiv.org/abs/2302.06675

13 14 Sign-Based Optimizers Are Effective Under Heavy-Tailed Noise
https://arxiv.org/html/2602.07425v1

15 16 AdaGC: Improving Training Stability for Large Language Model Pretraining
https://arxiv.org/html/2502.11034v1

17 18 47 49 AGGC: Adaptive Group Gradient Clipping for Stabilizing Large Language Model Training
https://arxiv.org/html/2601.11864v1

19 20 41 46 Generalized Gradient Norm Clipping & Non-Euclidean $(_{0},_{1})$-Smoothness
https://arxiv.org/html/2506.01913v2

21 Efficient Riemannian Optimization on the Stiefel Manifold via the Cayley Transform | OpenReview
https://openreview.net/forum?id=HJxV-ANKDH

22 Retraction-free optimization over the Stiefel manifold with application to the LoRA fine-tuning | OpenReview
https://openreview.net/forum?id=GP30inajOt&referrer=%5Bthe%20profile%20of%20Jiang%20Hu%5D(%2Fprofile%3Fid%3D~Jiang_Hu2)

23 NeurIPS Poster Spectral Conditioning of Attention Improves Transformer Performance
https://neurips.cc/virtual/2025/loc/san-diego/poster/118049

24 25 proceedings.neurips.cc
https://proceedings.neurips.cc/paper_files/paper/2023/file/d073692637b4fb8c4eb4b81f0fa2df7b-Paper-Conference.pdf

26 27 MSign: An Optimizer Preventing Training Instability in Large Language Models via Stable Rank Restoration
https://arxiv.org/html/2602.01734v1

28 29 MuonClip Optimizer for LLM Stability
https://www.emergentmind.com/topics/muonclip-optimizer

30 31 40 Optimizing Optimizers for Fast Gradient-Based Learning
https://arxiv.org/html/2512.06370v1

37 [2412.13148] SWAN: SGD with Normalization and Whitening Enables Stateless LLM Training
https://arxiv.org/abs/2412.13148

38 39 Gradient Multi-Normalization for Efficient LLM Training | OpenReview
https://openreview.net/forum?id=oanhUGY6un